

LANGGRAPH · QUICK REFERENCE

30 LangGraph Interview Q&A

Nodes, edges, state, checkpoints, and routing.
The concepts AI engineers actually use day to day.

30

Q&A Pairs

Swipe to start →



Ritesh Rai

GenAI Engineer

1

What is LangGraph?

Ans- LangGraph is a framework for building AI agents and workflows as a graph instead of one straight line of code. It's designed for cases that need loops, branching, and the ability to pause and resume work.

2

How is LangGraph different from LangChain?

Ans- LangChain helps you chain components like prompts, models, and retrievers into a mostly linear pipeline. LangGraph sits on top and adds a stateful graph layer, so the same kind of application can loop, branch, and recover from failures.

3

What is a StateGraph?

Ans- A StateGraph is the main class LangGraph uses to build a workflow. It lets you define nodes and edges that all read from and write to one shared state object as data moves through the graph.

```
from langgraph.graph import StateGraph
graph = StateGraph(AgentState)
```



4

How do you define state in LangGraph?

Ans- State in LangGraph is just a typed dictionary listing every field your nodes will read or write. You define it once as a TypedDict, then pass that type in when you create the StateGraph.

```
from typing import TypedDict
class AgentState(TypedDict):
    question: str
    answer: str
```

5

How do you add a node to a graph?

Ans- A node is one single step of work, written as a normal Python function. You register it on the graph with `add_node`, giving it a name and the function that should run whenever that node executes.

```
def retrieve(state: AgentState):
    return {"answer": "..."}
graph.add_node("retrieve", retrieve)
```



6

How do you add an edge between two nodes?

Ans- An edge is a fixed connection that decides which node runs next once the current one finishes. You create it with `add_edge`, passing in the source node name and the destination node name.

```
graph.add_edge("retrieve", "generate")
```

7

What are START and END in LangGraph?

Ans- START and END are built-in markers, not real nodes, that tell LangGraph where a workflow begins and where it must stop. Every graph needs at least one edge leading out of START and one edge leading into END.

```
from langgraph.graph import START, END
graph.add_edge(START, "retrieve")
graph.add_edge("generate", END)
```

8

How do you add conditional routing?

Ans- Conditional routing lets the graph pick a different next step at runtime instead of always following the same fixed path. You set it up with `add_conditional_edges`, which takes a router function that looks at the state and returns the name of the next node.

```
graph.add_conditional_edges(
    "retrieve",
    lambda s: "generate" if s["answer"] else "retry")
```



9

How do you compile a graph before running it?

Ans- Compiling turns your graph definition into something you can actually run. Calling `.compile()` checks that your nodes and edges are wired together correctly and hands back a runnable app object.

```
app = graph.compile()
```

10

How do you invoke a compiled LangGraph app?

Ans- `invoke()` is how you run a compiled graph from start to finish. You pass it the initial state as a dictionary, it executes each node in order until it reaches `END`, and it returns the final state.

```
result = app.invoke({"question": "Hi"})
```

11

What is a Checkpointer in LangGraph?

Ans- A checkpointer is what gives a graph memory across steps and runs. It automatically saves the state after every node completes, which is what lets a workflow be paused, recovered after a crash, or picked up again later.

```
from langgraph.checkpoint.memory import MemorySaver  
checkpointer = MemorySaver()
```



12

How do you enable persistence on a graph?

Ans- Persistence means a workflow's progress is actually saved instead of being lost the moment something interrupts it. You turn it on by passing your checkpointer into `compile()`, and every run is then stored under its own thread ID.

```
app = graph.compile(checkpointer=checkpointer)
```

13

What is `thread_id` used for in LangGraph?

Ans- A `thread_id` is simply a unique label for one specific conversation or workflow run. The checkpointer uses this ID to know exactly which saved state to load when that run continues later.

```
config = {"configurable": {"thread_id": "user-42"}}
app.invoke(input, config=config)
```

14

How do you stream a LangGraph execution?

Ans- Streaming lets you see a workflow's progress as it happens instead of waiting for the entire run to finish. Calling `.stream()` instead of `.invoke()` yields an update every time a node completes.

```
for update in app.stream({"question": "Hi"}):
    print(update)
```



15

What are Reducers in LangGraph state?

Ans- A reducer tells LangGraph how to combine updates from different nodes into the same state field, instead of one update simply overwriting another. A common example is appending new messages to a list rather than replacing it entirely.

```
from typing import Annotated
import operator
messages: Annotated[list, operator.add]
```

16

How do you pause a graph for human approval?

Ans- This is how you build a human approval step directly into a workflow. Compiling with `interrupt_before` and a node name tells LangGraph to stop right before that node runs, until a person allows it to continue.

```
app = graph.compile(
    checkpointer=checkpointer,
    interrupt_before=["execute_payment"])
```

17

How do you resume an interrupted graph?

Ans- Resuming means continuing a paused workflow exactly where it left off, not restarting it. You call `.invoke()` again with the same `thread_id` and no new input, and LangGraph picks up from the last saved checkpoint.

```
app.invoke(None, config=config)
```



18

What is a Supervisor pattern in a multi-agent LangGraph?

Ans- A supervisor is a single coordinating node inside a multi-agent graph. It looks at the incoming request, decides which specialized agent node should handle it, then collects and returns that agent's result.

19

How do you build a Router node?

Ans- A router is just a plain Python function, not a special LangGraph object. It reads the current state and returns a string matching one of your node names, and `add_conditional_edges` uses that return value to decide the next node.

```
def router(state):  
    return "sql_agent" if state["is_sql"] else "rag_agent"
```

20

How do you loop back to an earlier node?

Ans- Looping means sending execution back to a node that already ran earlier, instead of only ever moving forward. You do this with a conditional edge whose router can return the name of that earlier node whenever the state isn't good enough yet.

```
graph.add_conditional_edges(  
    "validate",  
    lambda s: "retrieve" if not s["ok"] else "end")
```



21

What is recursion_limit in LangGraph?

Ans- recursion_limit is a built-in safety cap on how many steps a graph is allowed to take during one run. If a workflow keeps looping past that limit, LangGraph raises an error instead of letting it run forever.

```
app.invoke(input, config={"recursion_limit": 25})
```

22

How do you pass runtime config like thread_id at invoke time?

Ans- Runtime settings such as thread_id are passed in through a config dictionary at invoke time, not hardcoded into the graph itself. Nodes and the checkpointer can both read values out of that same config while the graph is running.

```
app.invoke(input,  
  config={"configurable": {"thread_id": "t1"}})
```

23

How do you read and update state inside a node?

Ans- Inside a node function, you receive the entire current state as its argument, but you don't have to return all of it. You only return the fields you actually want to change, and LangGraph automatically merges them into the rest of the state.

```
def generate(state: AgentState):  
    return {"answer": call_llm(state["question"])}
```



24

add_edge vs add_conditional_edges, what's the difference?

Ans- add_edge always sends execution to the same fixed node next, no matter what happened before. add_conditional_edges instead runs a router function at execution time and picks the next node dynamically based on the current state.

25

How do you let a node call external tools?

Ans- ToolNode is a ready-made node that LangGraph provides specifically for running tools. When an earlier LLM node decides to call a tool, ToolNode actually executes it and writes the result back into state as a tool message.

```
from langgraph.prebuilt import ToolNode
graph.add_node("tools", ToolNode(tools))
```

26

How do you build a ReAct-style agent quickly?

Ans- create_react_agent is a shortcut that saves you from wiring an agent graph by hand. You give it your LLM and a list of tools, and it hands back an already-compiled graph that implements the reason-then-act loop for you.

```
from langgraph.prebuilt import create_react_agent
agent = create_react_agent(llm, tools)
```



27

How do you visualize a LangGraph workflow?

Ans- LangGraph can generate an actual picture of your workflow so you can see how the nodes and edges connect. Calling `get_graph()` on a compiled app and rendering it as a Mermaid diagram or PNG produces that visual.

```
app.get_graph().draw_mermaid_png()
```

28

How do you handle a node that fails intermittently?

Ans- A node that depends on something unreliable, like an external API, shouldn't be allowed to crash the whole graph when it fails. Wrap the risky call in a `try/except`, store an error flag in state, and use a conditional edge to route to a retry or fallback node instead.

29

What is MessagesState in LangGraph?

Ans- `MessagesState` is a ready-made state schema built specifically for chat-style agents. It gives you a single `messages` field that automatically appends new messages instead of overwriting the whole conversation history.

```
from langgraph.graph import MessagesState
graph = StateGraph(MessagesState)
```



30

How would you debug a LangGraph workflow in production?

Ans- Start from the checkpointed state history for the specific thread_id that failed, since it shows exactly which node ran last and what its inputs and outputs were. Only dig into routing logic once you actually know which single node misbehaved.



That's a wrap.

30/30 LangGraph Q&A

Save this for your next AI Agent interview,
and keep it handy while shipping your next workflow.

Follow for more, and comment
“LANGGRAPH” for the full breakdown



Ritesh Rai, GenAI Engineer

300K+ LinkedIn impressions · Top 1% Creator on Topmate